Penetration Testing Report

Full Name: EMMANUEL ATEJI
Program: HCPT
Date: 18TH FEBRURARY 2025

Introduction

This report document hereby describes the proceedings and results of a Black Box security assessment conducted against the **Week 1 Labs**. The report hereby lists the findings and corresponding best practice mitigation actions and recommendations.

1. Objective

The objective of the assessment was to uncover vulnerabilities in the **Week 1 Labs** and provide a final security assessment report comprising vulnerabilities, remediation strategy and recommendation guidelines to help mitigate the identified vulnerabilities and risks during the activity.

2. Scope

This section defines the scope and boundaries of the project.

Application Name	Lab 1 - HTML Injection Lab 2 - Cross Site Scripting

3. Summary

Outlined is a Black Box Application Security assessment for the **Week {1} Labs**.

Total number of Sub-labs: {count} Sub-labs

High	Medium	Low
{4}	{4}	{9}

High - **4 Sub-labs with high difficulty level**

Medium - **4 Sub-labs with Medium difficulty level**

Low - **9 Sub-labs with low difficulty level**

1. HTML Injection

1.1. HTML's are easy!

Reference	Risk Rating
Sub-lab-1: HTML's are easy!	**Low**

Tools Used

Browser "Inspector" is used to find the vulnerability.

Vulnerability Description

HTML Injection is a web security vulnerability that occurs when user-supplied input is improperly handled, allowing HTML code to be injected into a webpage. In this case, the vulnerability exists in the **search functionality**, where user input is **dynamically reflected** on the page without proper sanitization or encoding. As a result, HTML tags, such as **<h1>,** are processed and rendered as actual elements instead of being treated as plain text.
This issue arises from improper input validation and output encoding, making the application susceptible to injection-based attacks.

Payload used: <h1>Hello World!</h1>

How It Was Discovered

Manual Analysis

Vulnerable URLs

https://labs.hacktify.in/HTML/html_lab/lab_1/index.php

Consequences of not Fixing the Issue

If the HTML Injection vulnerability is not patched, it can lead to several security risks, including:

1. **Webpage Defacement** – Attackers can inject arbitrary HTML to modify the appearance of the webpage, potentially displaying misleading or malicious content.
2. **Phishing Attacks** – Malicious actors can inject fake login forms or deceptive messages to trick users into providing sensitive information such as usernames, passwords, or financial details.
3. **Client-Side Code Execution** – If combined with JavaScript execution (via event handlers like <script>, <img onerror>, etc.), this could escalate to **Cross-Site Scripting (XSS)**, allowing attackers to execute malicious scripts in the victim's browser.
4. **Session Hijacking & Account Takeover** – If JavaScript injection is possible, attackers can steal session cookies, enabling them to impersonate users and gain unauthorized access.
5. **Redirection to Malicious Sites** - Attackers can inject <meta> refresh tags or JavaScript to redirect users to phishing or malware-infected sites.

Suggested Countermeasures

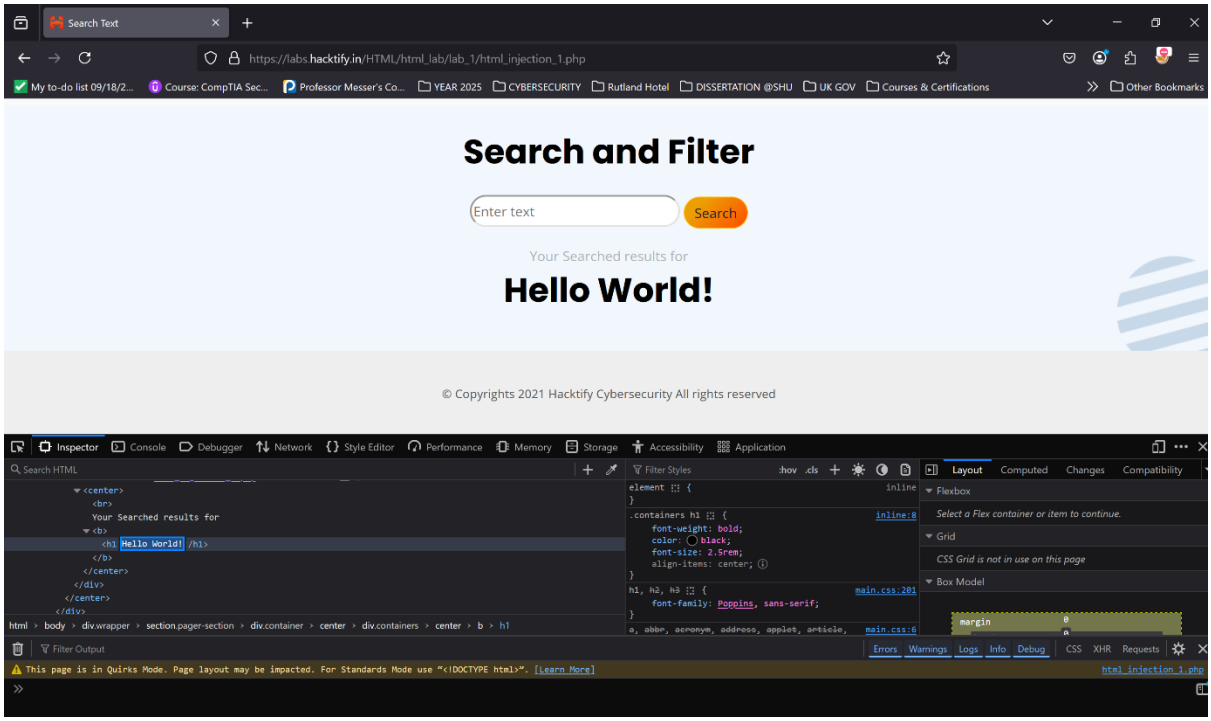
- **Output Encoding**: Before reflecting user input on the webpage, encode special characters to prevent them from being interpreted as HTML.
- **Proper Input Validation** – Restrict user input to **only allow expected characters** (e.g., letters, numbers) and reject HTML tags.
- **Content Security Policy (CSP)** – Implement a **CSP header** to prevent the browser from executing injected scripts, reducing the risk of script-based attacks:

References

https://portswigger.net/web-security/html-injection
https://developer.mozilla.org/en-US/docs/Web/Security/HTML_Injection
https://owasp.org/www-community/attacks/HTML_Injection
https://www.acunetix.com/websitesecurity/html-injection/
https://www.imperva.com/learn/application-security/html-injection/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



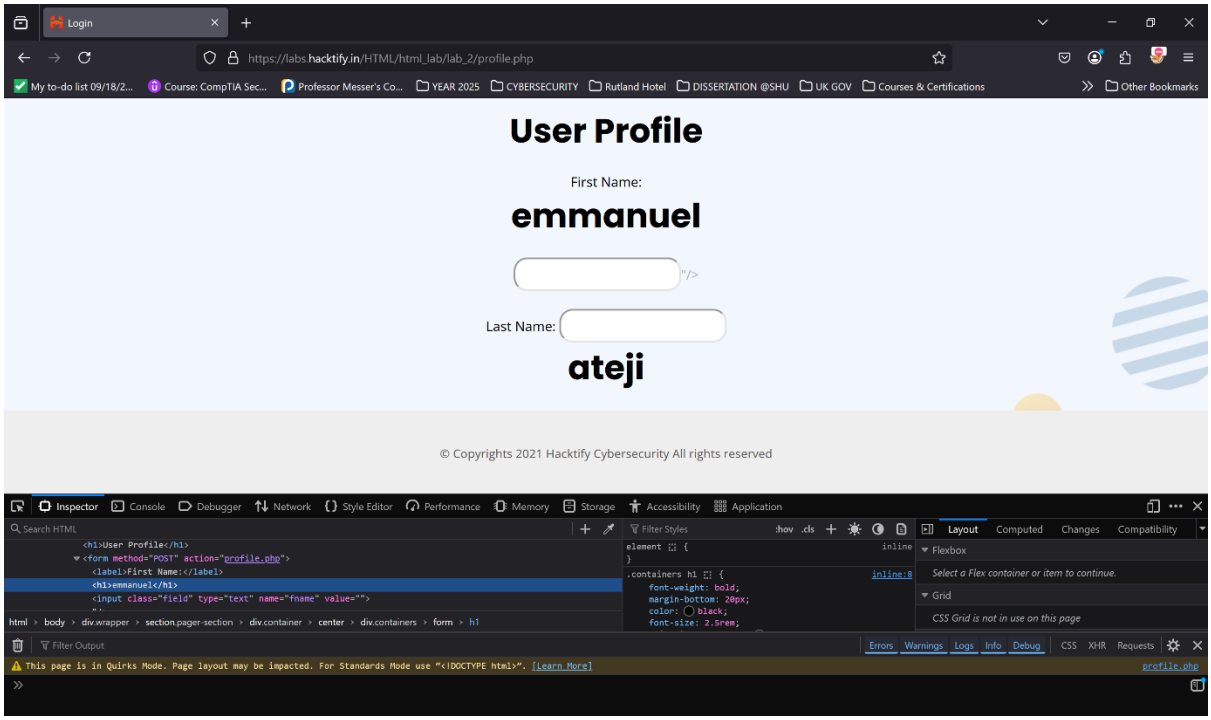
1.2. Let me Store them!

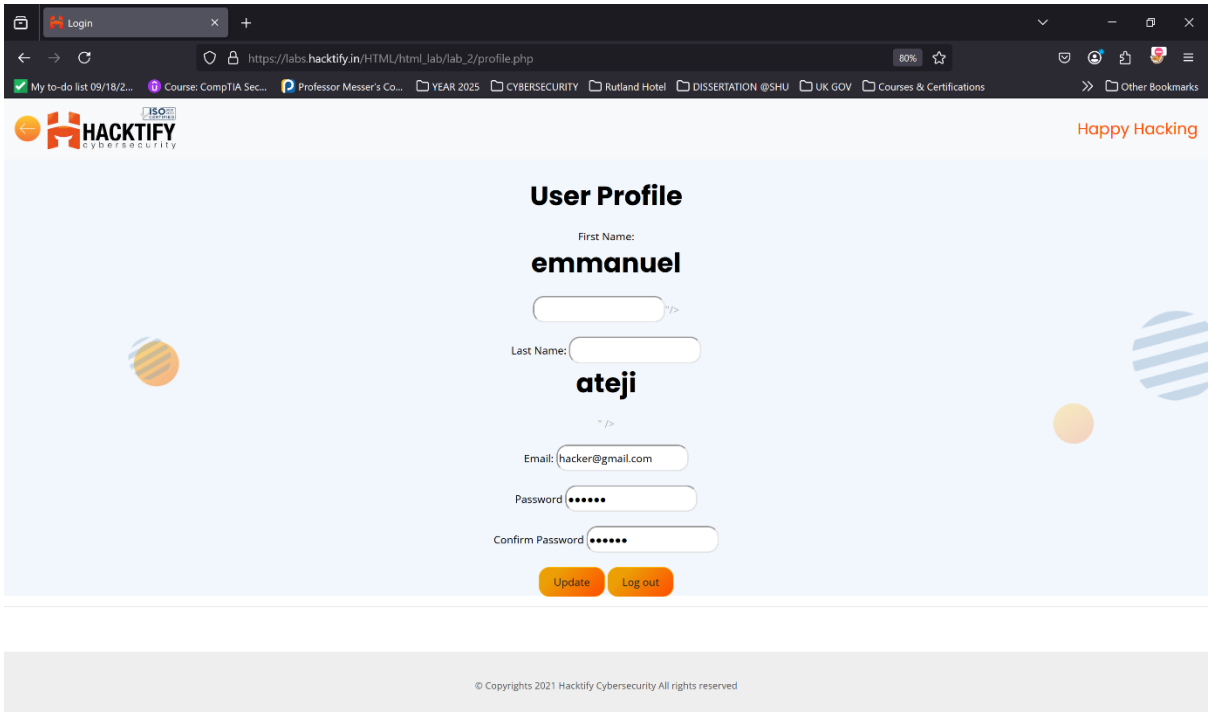
Reference	Risk Rating
Sub-lab-2: Let me Store them!	**Low**
Tools Used	
Browser “Inspector” is used to find the vulnerability.	
Vulnerability Description	
HTML Injection is a web security vulnerability that occurs when an application allows user input to be injected into an HTML page without proper validation or sanitization. This can allow attackers to manipulate the HTML structure, insert unwanted scripts, or alter the webpage’s behavior. In this case, this indicates that the application improperly handles user input, allowing for HTML elements to be injected into profile fields. Additionally, it appears that passwords may be stored improperly, suggesting potential improper storage mechanisms, increasing the risk of credential exposure. Payload: "/><h1>emmanuel</h1> "/><h1>ateji</h1>	
How It Was Discovered	

Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/html_lab/lab_2/index.php
Consequences of not Fixing the Issue
1. **Content Manipulation:** Attackers can alter the appearance and content of the page, leading to misinformation or defacement. 2. **Phishing Attacks:** Malicious actors can inject fake login forms to steal user credentials. 3. **Session Hijacking or Data Theft:** If combined with JavaScript injection (Cross-Site Scripting - XSS), attackers can steal session cookies or sensitive information. 4. **Reputation Damage:** If exploited, this vulnerability can harm the credibility and trustworthiness of the website.
Suggested Countermeasures
1. **Input Validation & Escaping:** Ensure that all user input is properly sanitized by escaping special HTML characters such as <, >, ", and ' to prevent injection. 2. **Use Content Security Policy (CSP):** Implement CSP headers to restrict the execution of malicious scripts injected into the page. 3. **Server-Side Validation:** Always validate and sanitize input on the server side, not just on the client side, to prevent malicious payloads from being stored and rendered later.
References
https://www.imperva.com/learn/application-security/html-injection/ https://www.acunetix.com/vulnerabilities/web/html-injection/ https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/11-Client-side_Testing/03-Testing_for_HTML_Injection https://www.invicti.com/learn/html-injection/ https://www.utep.edu/information-resources/iso/security-awareness/technical-security-resources/what-is-html-injection.html

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab





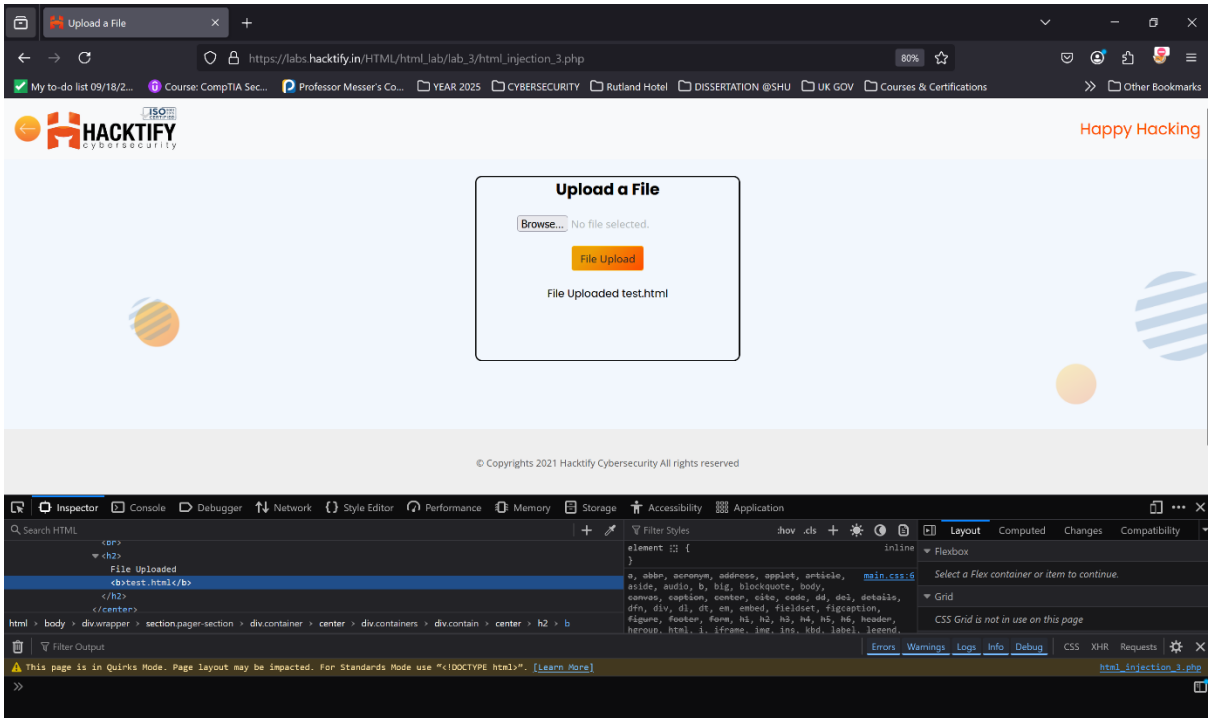
1.3. File Names are also vulnerable!

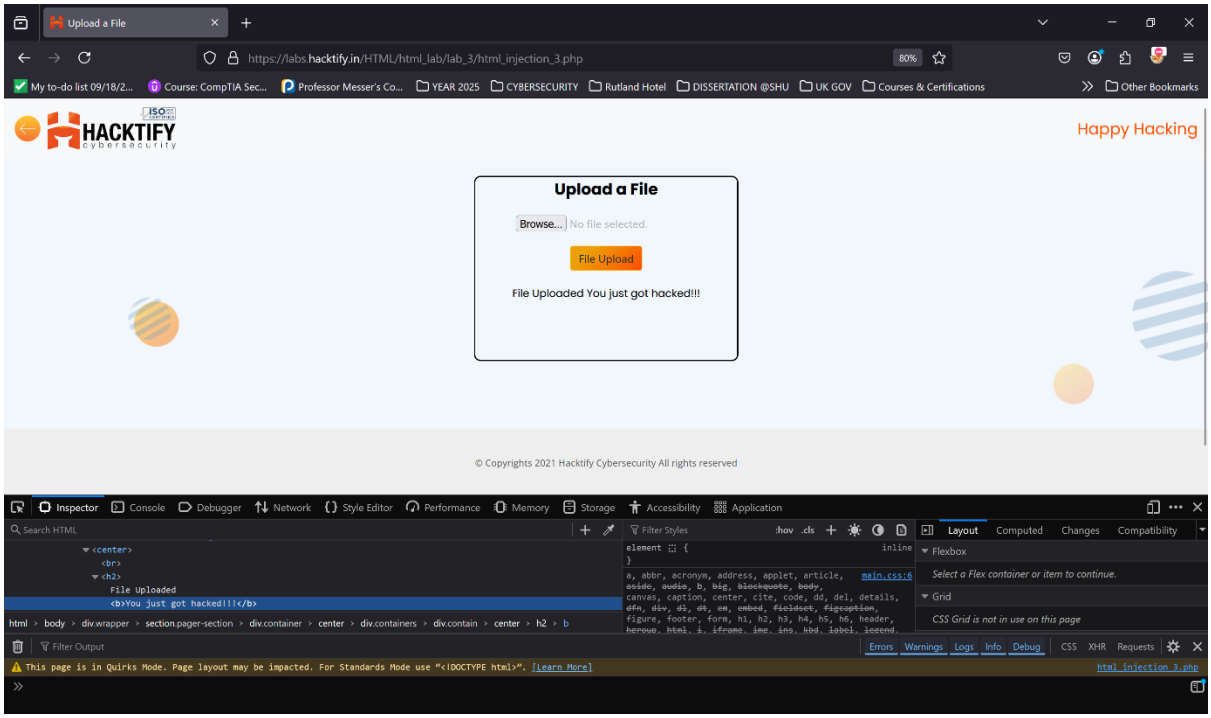
Reference	Risk Rating
Sub-lab-3: File Names are also vulnerable!	**Low**
Tools Used	
Browser "Inspector" is used to find the vulnerability.	

Vulnerability Description
The file upload functionality is vulnerable to HTML injection through malicious filenames. The application displays the uploaded filename directly in the HTML output without proper sanitization or encoding. This allows attackers to inject arbitrary HTML tags and content through specially crafted filenames, which are then interpreted and rendered by the browser. This shows how a filename containing HTML tags (like <b> tags) is rendered as formatted HTML rather than plain text.
How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/html_lab/lab_3/html_injection_3.php
Consequences of not Fixing the Issue
1. **Cross-Site Scripting (XSS) Attacks** - Attackers can inject malicious JavaScript through filenames to steal session cookies, hijack user sessions, or perform actions on behalf of victims. 2. **Content Spoofing -** Malicious users could inject HTML to modify page content and interface elements, potentially misleading users or redirecting them to phishing sites. 3. **Data Exfiltration -** Through injected scripts, attackers could access and steal sensitive data from the application or user's browser. 4. **Website Defacement** - The ability to inject HTML allows attackers to modify the visual appearance and content of the page, damaging site reputation.
Suggested Countermeasures
1. Implement **proper HTML encoding** of filenames before displaying them on the page using functions like htmlspecialchars() in PHP or equivalent encoding functions in other languages. 2. Add **content security policies (CSP)** to restrict execution of injected scripts and loading of external resources. 3. **Validate and sanitize** filenames to only allow safe characters, removing or blocking any HTML tags or special characters.
References
https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload https://portswigger.net/web-security/cross-site-scripting https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html https://www.acunetix.com/websitesecurity/upload-forms-threats/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab





1.4. File Content and HTML Injection a perfect pair!

Reference	Risk Rating
Sub-lab-4: File Content and HTML Injection a perfect pair!	**Medium**
Tools Used	
Web Browser, Sublime Text Editor is used to find this vulnerability	
Vulnerability Description	

The web application allows unrestricted file uploads and executes HTML/JavaScript content from uploaded files. This vulnerability occurs when the application fails to properly validate file types and content, allowing attackers to upload malicious HTML files containing JavaScript code. In this case, the application accepted and executed a test.html file containing document.cookie script, which revealed sensitive cookie information, demonstrating a combined file upload and HTML injection vulnerability.
How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/html_lab/lab_4/html_injection_4.php
Consequences of not Fixing the Issue
1. **Cross-Site Scripting (XSS) Attacks**: Attackers can upload files containing malicious JavaScript to steal user sessions, cookies, and sensitive data. 2. **Session Hijacking**: As demonstrated by accessing the cookies, attackers can capture session tokens to impersonate legitimate users. 3. **Data Exfiltration**: Malicious scripts can be used to harvest sensitive information from the application and send it to attacker-controlled servers. 4. **Web Shell Upload**: The file upload vulnerability could potentially be leveraged to upload more severe payloads like web shells for remote server access.
Suggested Countermeasures
1. Implement strict file type validation and whitelist only allowed file extensions, particularly blocking executable content like .html, .js, and other script files. 2. Apply content-type verification and scan uploaded files for malicious code or scripts before processing them. 3. Store uploaded files outside of the web root and serve them through a secure download script that sets appropriate Content-Disposition headers.
References
https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload https://portswigger.net/web-security/file-upload https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/10-Business_Logic_Testing/09-Test_Upload_of_Malicious_Files

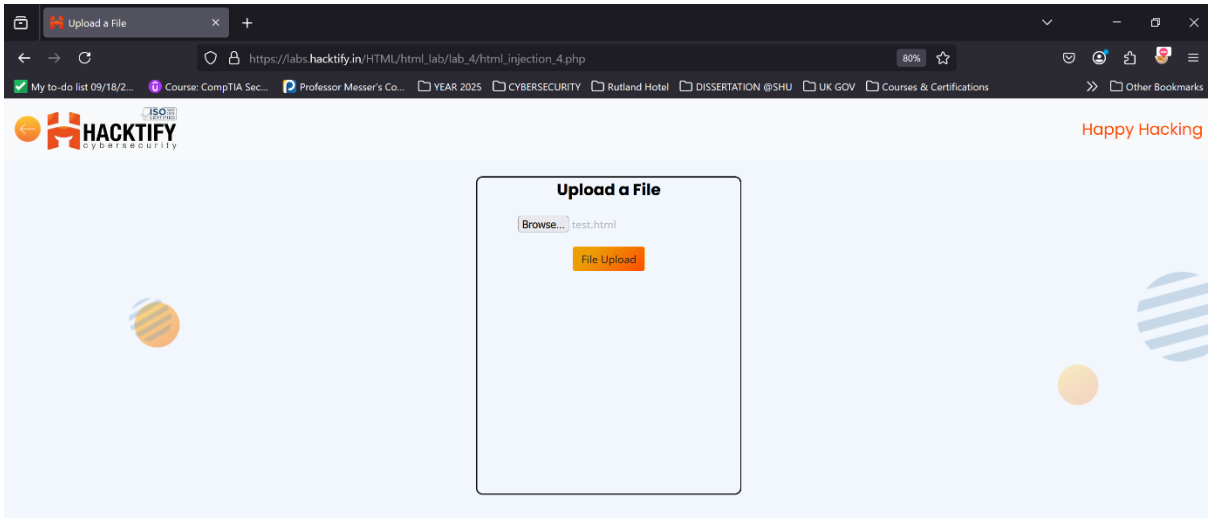
Proof of Concept

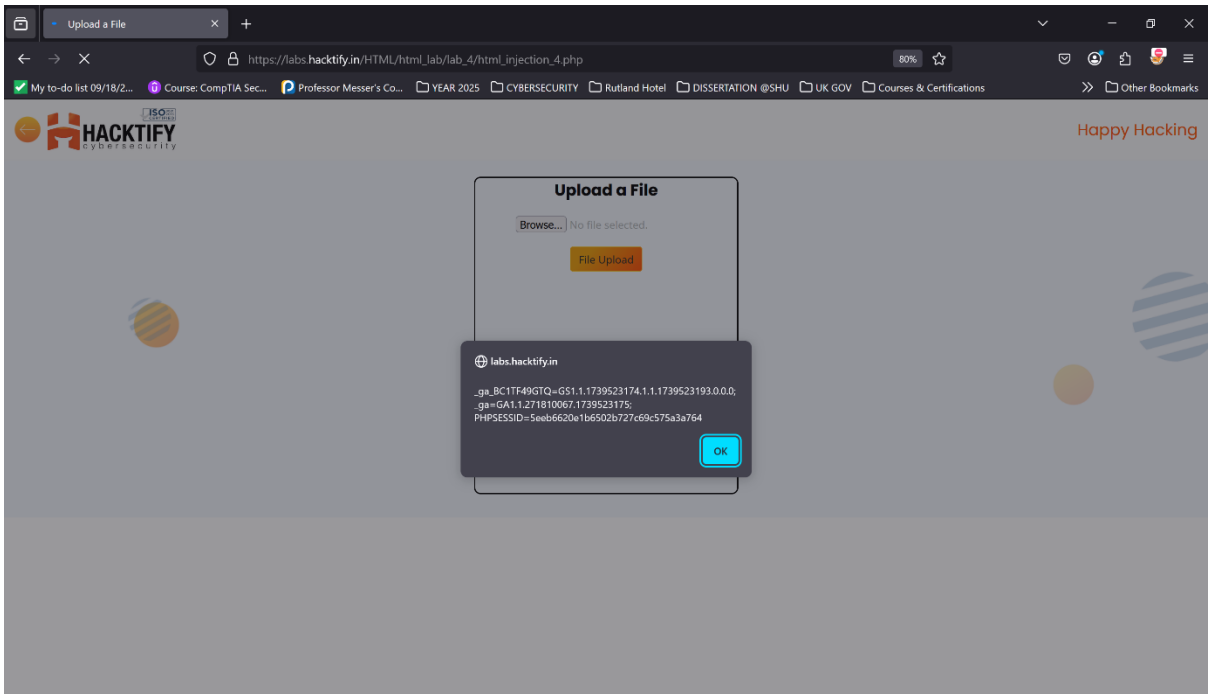
This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

File Edit Selection Find View Goto Tools Project Preferences Help

Codes.html | test.html

```
1 <script>alert(document.cookie)</script>
2
```



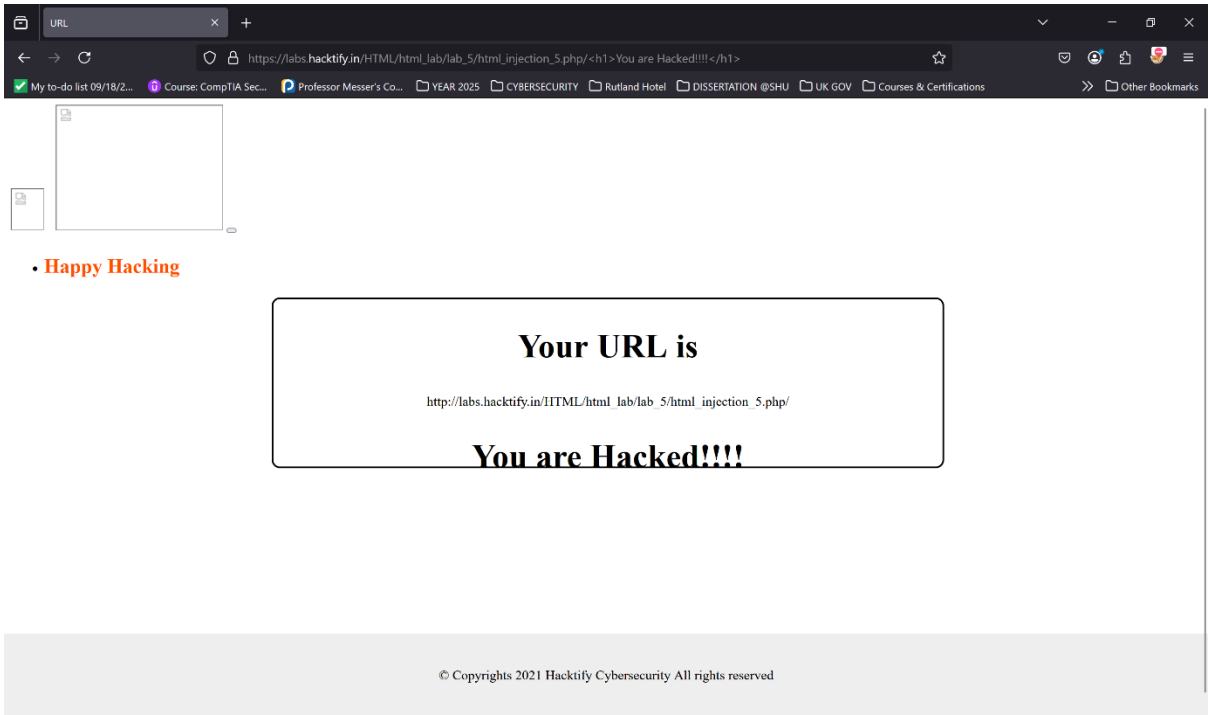


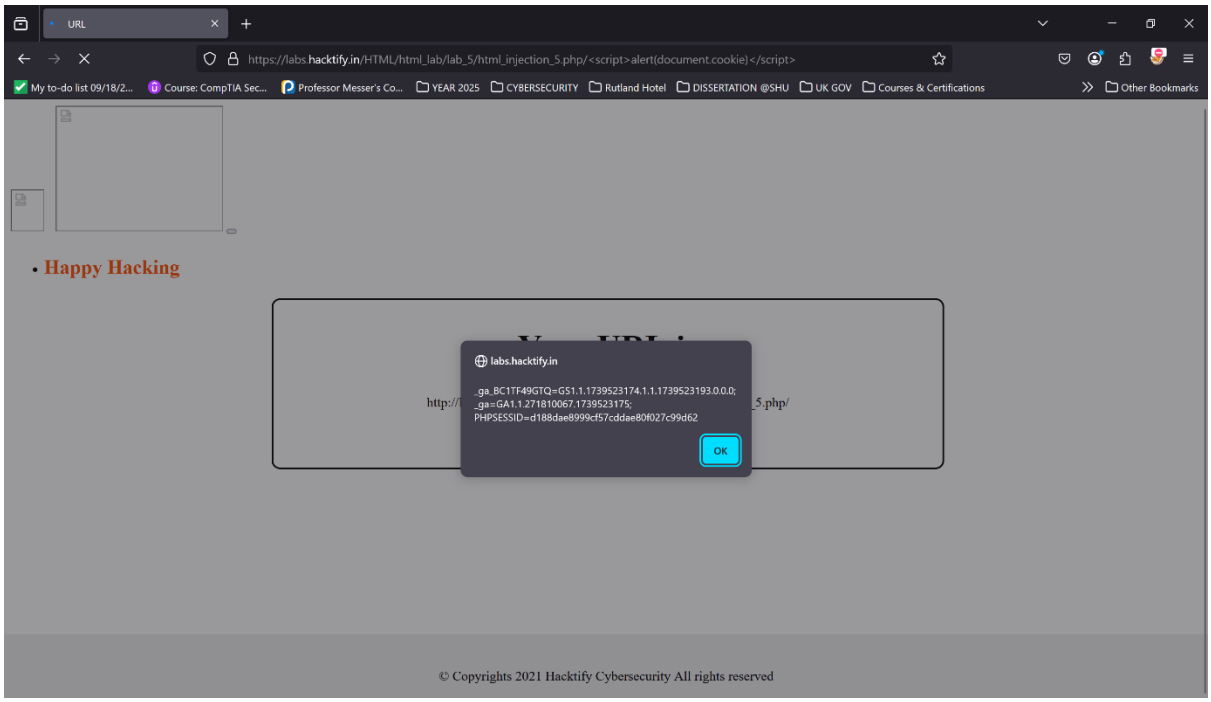
1.5. Injecting HTML using URL

Reference	Risk Rating
Sub-lab-5: Injecting HTML using URL	**Medium**
Tools Used	
Web Browser is used to find the vulnerability.	
Vulnerability Description	
HTML Injection (also known as markup injection) is a vulnerability that occurs when an application accepts unvalidated user input which includes HTML markup or scripts that are then rendered in a web page. In this instance, the application is directly accepting and rendering HTML/JavaScript code passed through the URL parameters without proper sanitization. This allows attackers to inject arbitrary HTML and JavaScript code that executes in users' browsers. Payload Used: <script>alert(document.cookie)</script> <h1>You are Hacked!!!</h1>	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/html_lab/lab_5/index.php	
Consequences of not Fixing the Issue	
1. **Cross-Site Scripting (XSS) Attacks** - Attackers can inject malicious JavaScript to steal sensitive data like cookies, session tokens, and other user information 2. **Content Spoofing** - Attackers can modify the visual appearance of the page to mislead users or conduct phishing attacks 3. **Session Hijacking** - Through cookie theft enabled by injected scripts, attackers can take over authenticated user sessions 4. **Browser Exploitation** - Malicious code injection could be used to exploit browser vulnerabilities and gain control of users' systems	
Suggested Countermeasures	
1. Implement proper input validation and sanitization - Use HTML encoding/escaping functions to convert special characters into their HTML entity equivalents before rendering user input 2. Apply Content Security Policy (CSP) headers to restrict the sources of content that can be loaded and executed in the web application 3. Use built-in framework security features or dedicated security libraries that provide protection against HTML injection	
References	
https://www.sans.org/blog/understanding-html-injection/ https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload https://portswigger.net/web-security/file-upload https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab





1.6 Encode IT!

Reference	Risk Rating
Sub-lab-6: Encode IT!	**High**
Tools Used	

Web Browser, Online URL encoder is used to find this vulnerability

Vulnerability Description

This is an HTML Injection vulnerability where the application accepts and processes user-supplied HTML/JavaScript code without proper sanitization. The vulnerability allows attackers to inject malicious scripts that can execute in the context of other users' browsers.
In this instance, we can see that the application accepts and processes a script tag that attempts to access document cookies, which is a common XSS payload for testing.

Payload Used: (<script>alert(document.cookie)</script>)

How It Was Discovered

Manual Analysis

Vulnerable URLs

https://labs.hacktify.in/HTML/html_lab/lab_6/index.php

Consequences of not Fixing the Issue

1. **Cookie Theft & Session Hijacking** - Attackers can steal user session cookies and impersonate legitimate users.
2. **Data Exfiltration** - Malicious scripts can access and transmit sensitive information from the user's browser.
3. **Malicious Actions** - Attackers can perform unauthorized actions on behalf of the victim user
4. **Browser Resource Exploitation** - Injected code can manipulate the DOM, redirect users, or exploit browser resources.

Suggested Countermeasures

Implement Context-Aware Output Encoding

- Use appropriate encoding functions (HTML, JavaScript, URL) based on where the user input is being rendered
- Ensure all user-supplied data is properly encoded before being reflected in the response

Input Validation & Sanitization

- Implement strict input validation to reject potentially malicious input
- Use security libraries like DOMPurify to sanitize HTML input when HTML input is required

Content Security Policy (CSP)

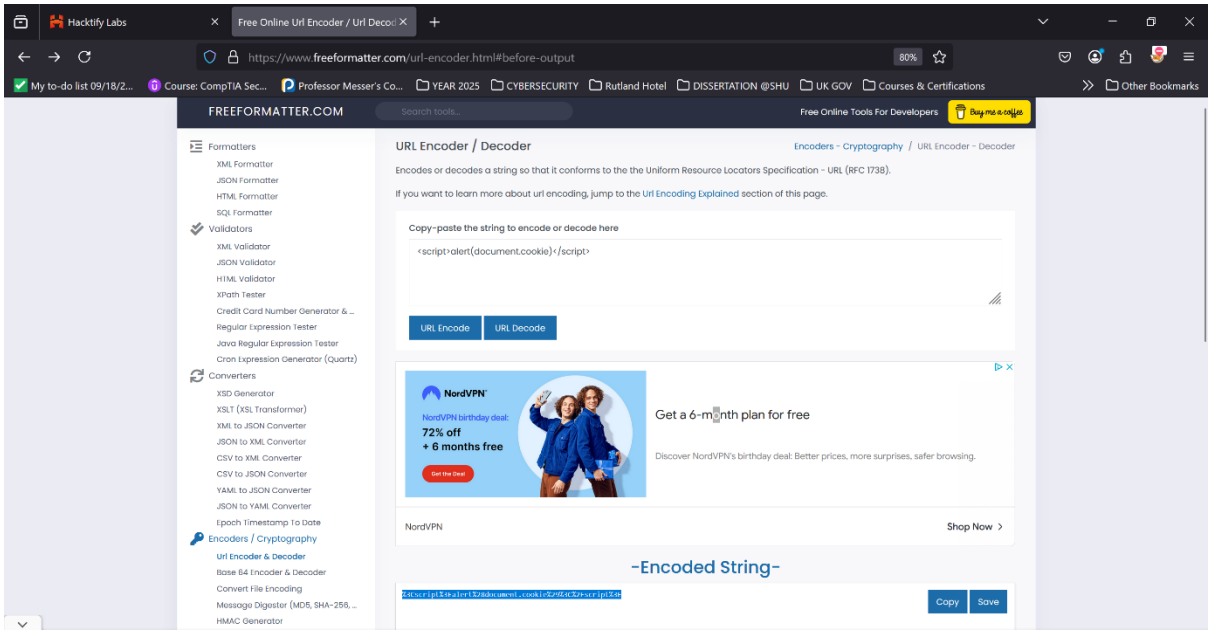
- Implement strict CSP headers to prevent execution of inline scripts and restrict sources of content

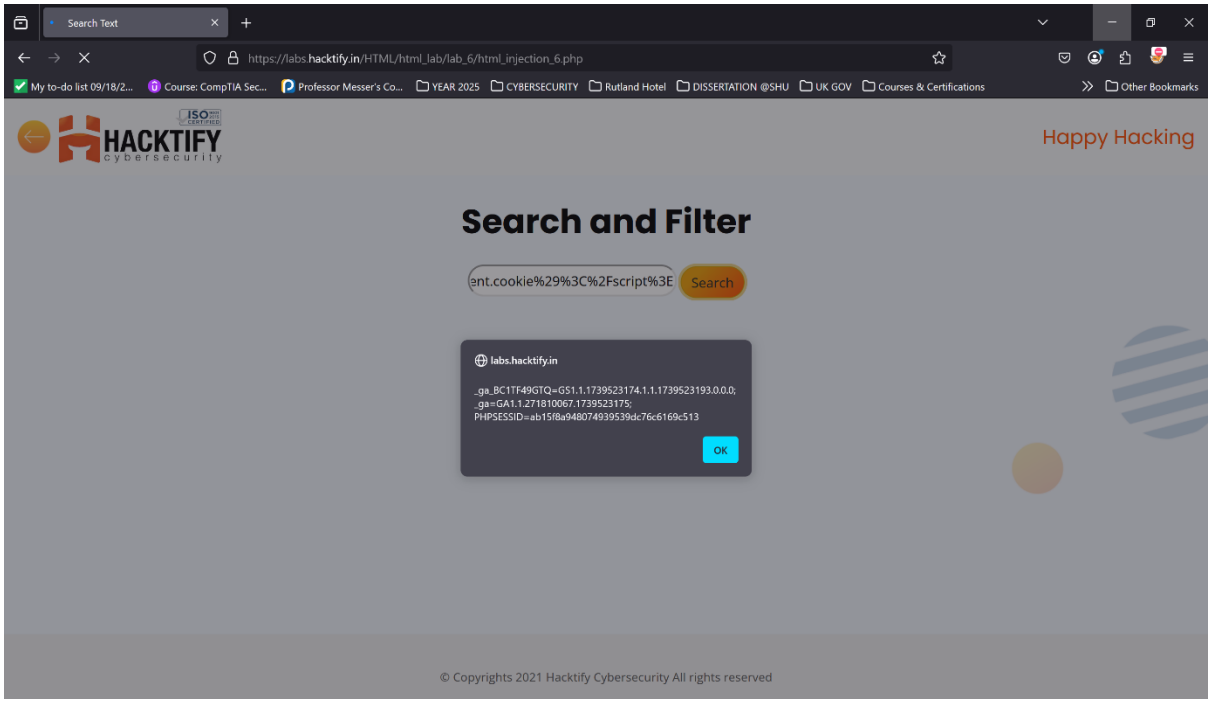
References

https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
https://portswigger.net/web-security/file-upload
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/10-Business_Logic_Testing/09-Test_Upload_of_Malicious_Files

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab





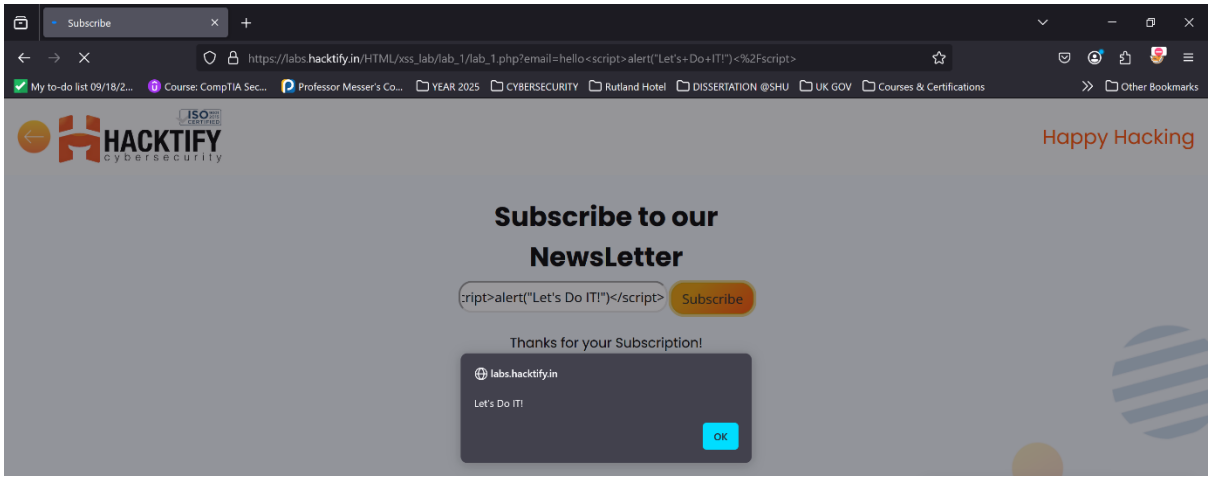
2. Cross Site Scripting

2.1. Let's Do IT!

Reference	Risk Rating
Sub-lab-1: Let's Do IT!	**Low**
Tools Used	
Web Browser is used to find this vulnerability	
Vulnerability Description	
This is a reflected Cross-Site Scripting (XSS) vulnerability where the application fails to properly sanitize user input in the email subscription form. The application accepts and executes arbitrary JavaScript code (<script>alert("Let's Do IT!")</script>) without proper input validation or encoding. This indicates that the application is directly reflecting user input into the page's HTML context without implementing proper security controls. Payload: <script>alert("Let's Do IT!")</script>	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
URLs of the vulnerable pages in the lab	
Consequences of not Fixing the Issue	
• Attackers can execute malicious JavaScript code in users' browsers, potentially stealing session cookies and hijacking user sessions • Ability to perform unauthorized actions on behalf of legitimate users by capturing their authentication credentials • Potential for data theft through form hijacking and keylogging capabilities • Website defacement and manipulation of page content, damaging brand reputation	
Suggested Countermeasures	
• Implement proper input validation and sanitization using HTML encoding/escaping functions for all user-supplied input before displaying it on the page • Use Content Security Policy (CSP) headers to restrict execution of unauthorized JavaScript code • Utilize built-in XSS protection frameworks or libraries like DOMPurify, and escape special characters using functions like htmlspecialchars()	
References	
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html https://www.sans.org/blog/cross-site-scripting-explained/	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



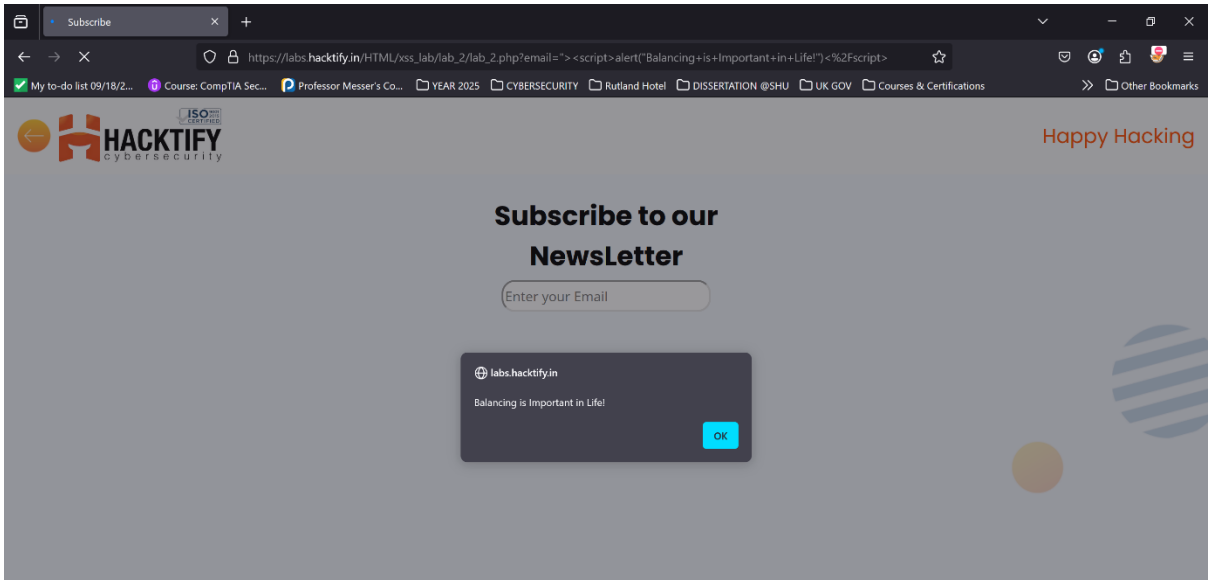
2.2. Balancing is Important in Life!

Reference	Risk Rating
Sub-lab-2: Balancing is Important in Life!	**Low**
Tools Used	
Web Browser was used for this vulnerability.	
Vulnerability Description	
This appears to be a Reflected XSS vulnerability in the email input field where user input is not properly sanitized. The application accepts and executes arbitrary JavaScript code injected through the URL parameter "email". In this case, the payload breaks out of the existing HTML context using "> and injects a JavaScript alert dialog. This indicates insufficient input validation and output encoding. Payload: "><script>alert("Balancing is Important in Life!")</script>	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/xss_lab/lab_2/lab_2.php	
Consequences of not Fixing the Issue	
1. **Session Hijacking** - Attackers can steal user session cookies and impersonate legitimate users. 2. **Data Theft** - Malicious scripts can access and exfiltrate sensitive user information 3. **Phishing Attacks** - Attackers can inject convincing fake forms to steal credentials 4. **Website Defacement** - Malicious code can modify the page content and compromise brand reputation	
Suggested Countermeasures	

1. Implement proper input validation and sanitization on the server-side to reject malicious characters and scripts. 2. Use context-aware output encoding (HTML encode special characters like < > " ') 3. Implement Content Security Policy (CSP) headers to restrict script execution sources.
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html https://www.sans.org/blog/cross-site-scripting-explained/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



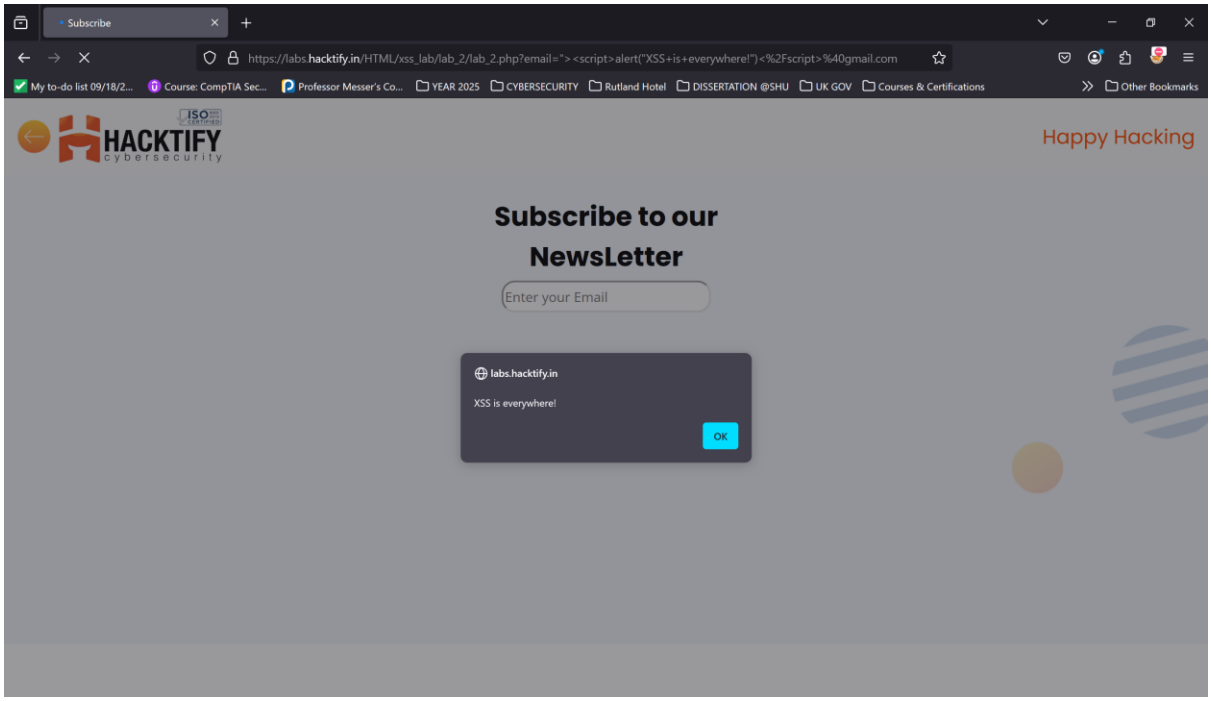
2.3. XSS is everywhere!

Reference	Risk Rating
Sub-lab-3: XSS is everywhere!	**Low**
Tools Used	
Web Browser is used to find this vulnerability	
Vulnerability Description	
This is a Reflected Cross-Site Scripting (XSS) vulnerability in the newsletter subscription form where the application fails to properly sanitize user input in the email field. The vulnerable parameter accepts and executes arbitrary JavaScript code injected through the email input field. The payload breaks out of the email input context using "> and then injects a script tag that executes an alert function.	

Payload used: "><script>alert("XSS is everywhere!")</script>@gmail.com
How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/xss_lab/lab_3/lab_3.php
Consequences of not Fixing the Issue
• **Session Hijacking** - Attackers could steal users' session cookies and impersonate them • **Data Theft** - Malicious scripts could capture sensitive user information and form data • **Phishing Attacks** - Attackers could inject convincing fake login forms to steal credentials • **Website Defacement** - Malicious JavaScript could modify page content and functionality
Suggested Countermeasures
• Implement proper input validation and sanitization for the email field using built-in functions or security libraries (e.g., DOMPurify). • Apply HTML encoding to special characters before rendering user input to prevent script execution. • Implement Content Security Policy (CSP) headers to restrict script execution sources.
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html https://www.sans.org/blog/cross-site-scripting/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



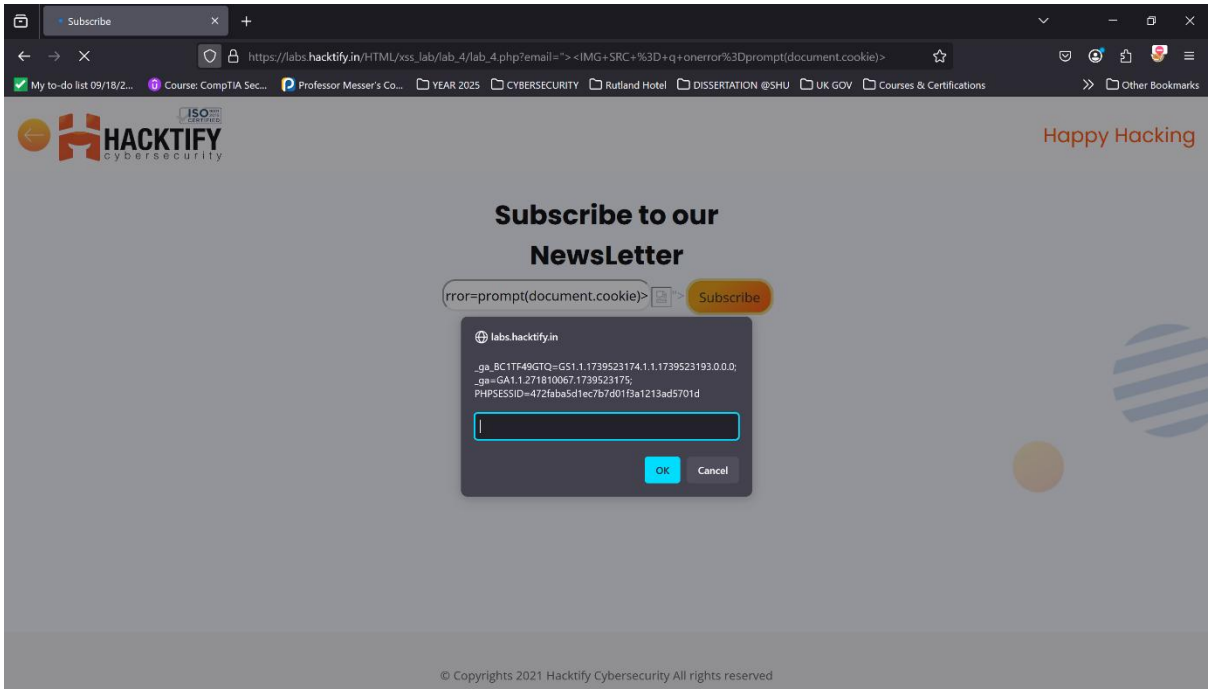
2.4. Alternatives are must!

Reference	Risk Rating
Sub-lab-4: Alternatives are must!	**Low**
Tools Used	
Web Browser is used to find this vulnerability	
Vulnerability Description	
This is a Reflected Cross-Site Scripting (XSS) vulnerability in the newsletter subscription form. The application fails to properly validate and sanitize user input, allowing an attacker to inject malicious JavaScript code. The payload "><IMG SRC = q onerror=prompt(document.cookie)> breaks out of the input context using ">, then injects an IMG tag with an invalid source, triggering the onerror event to execute arbitrary JavaScript (in this case, accessing document cookies). Payload: "><IMG SRC = q onerror=prompt(document.cookie)>	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/xss_lab/lab_3/index.php	
Consequences of not Fixing the Issue	
• **Cookie/Session Theft** - Attackers can steal user session cookies and hijack authenticated sessions • **Data Exfiltration** - Sensitive user information could be captured and transmitted to malicious servers • **Account Compromise** - Attackers could perform actions on behalf of the victim user • **Malicious Content Injection** - Attackers could modify page content or redirect users to malicious sites	
Suggested Countermeasures	

• Implement proper input validation and sanitization by escaping special characters and HTML entities before rendering user input. • Use Content Security Policy (CSP) headers to restrict execution of inline scripts and specify trusted sources. • Apply context-aware output encoding when rendering user-supplied data in HTML contexts.
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html https://www.sans.org/blog/cross-site-scripting-explained/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



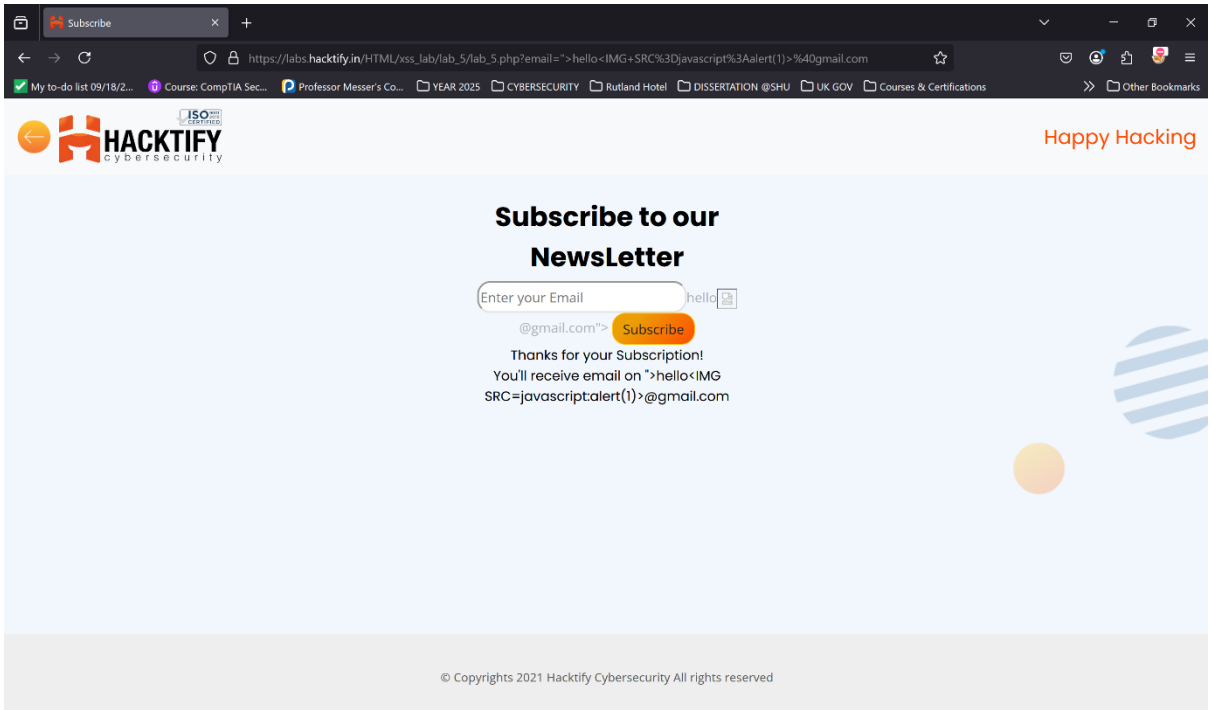
2.5. Developer hates scripts!

Reference	Risk Rating
Sub-lab-5: Developer hates scripts!	**High**
Tools Used	
Web Browser is used to find the vulnerability.	
Vulnerability Description	
This is a Reflected Cross-Site Scripting (XSS) vulnerability in the newsletter subscription form. The application fails to properly validate and sanitize email input, allowing an attacker to inject malicious	

HTML and JavaScript code. The payload ">hello<IMG SRC=javascript:alert(1)>@gmail.com" breaks out of the email input field's context and executes arbitrary JavaScript code through the IMG tag's SRC attribute.
How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/xss_lab/lab_5/lab_5.php
Consequences of not Fixing the Issue
• **Session Hijacking** - Attackers can steal users' session cookies and impersonate them • **Data Theft** - Malicious scripts can capture sensitive user information and form inputs • **Phishing Attacks** - Attackers can inject convincing fake forms or content to steal credentials • **Website Defacement** - The vulnerability allows injection of unauthorized content, potentially damaging the site's reputation
Suggested Countermeasures
1. **Implement proper input validation and sanitization**: • Use email-specific validation regex patterns • HTML-encode special characters before rendering user input 2. Implement Content Security Policy (CSP) headers to restrict execution of inline scripts and prevent unauthorized source loading 3. Use built-in framework protection mechanisms like Angular's DomSanitizer or React's ESLint rules to automatically escape dangerous content
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html https://www.sans.org/blog/cross-site-scripting-for-dummies/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



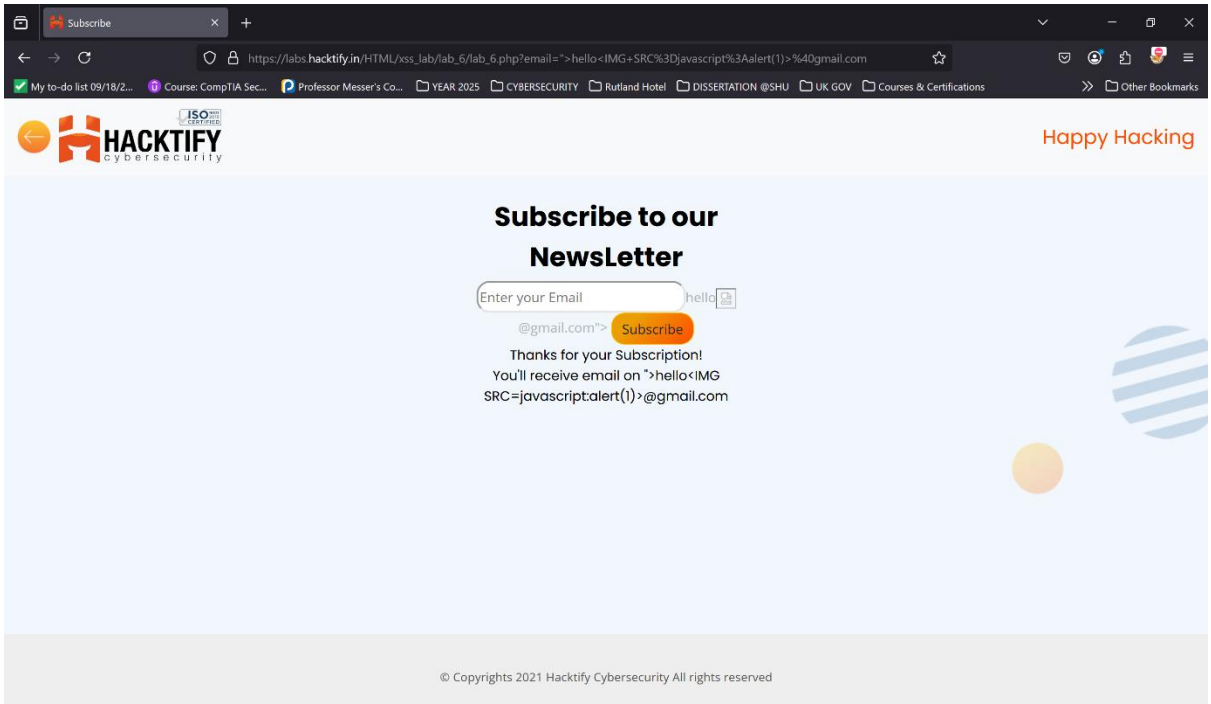
2.6. Change the Variation!

Reference	Risk Rating
Sub-lab-6: Change the Variation!	**High**
Tools Used	
Web Browser is used to find the vulnerability.	
Vulnerability Description	
This is a Reflected Cross-Site Scripting (XSS) vulnerability in the newsletter subscription form. The application fails to properly validate and sanitize email input, allowing an attacker to inject malicious HTML and JavaScript code. The payload ">hello<IMG SRC=javascript:alert(1)>@gmail.com" breaks out of the email input field's context and executes arbitrary JavaScript code through the IMG tag's SRC attribute.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/xss_lab/lab_6/index.php	
Consequences of not Fixing the Issue	
• **Session Hijacking** - Attackers can steal users' session cookies and impersonate them • **Data Theft** - Malicious scripts can capture sensitive user information and form inputs • **Phishing Attacks** - Attackers can inject convincing fake forms or content to steal credentials • **Website Defacement** - The vulnerability allows injection of unauthorized content, potentially damaging the site's reputation	
Suggested Countermeasures	
1. **Implement proper input validation and sanitization**: • Use email-specific validation regex patterns • HTML-encode special characters before rendering user input	

2. Implement Content Security Policy (CSP) headers to restrict execution of inline scripts and prevent unauthorized source loading 3. Use built-in framework protection mechanisms like Angular's DomSanitizer or React's ESLint rules to automatically escape dangerous content
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html https://www.sans.org/blog/cross-site-scripting-for-dummies/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



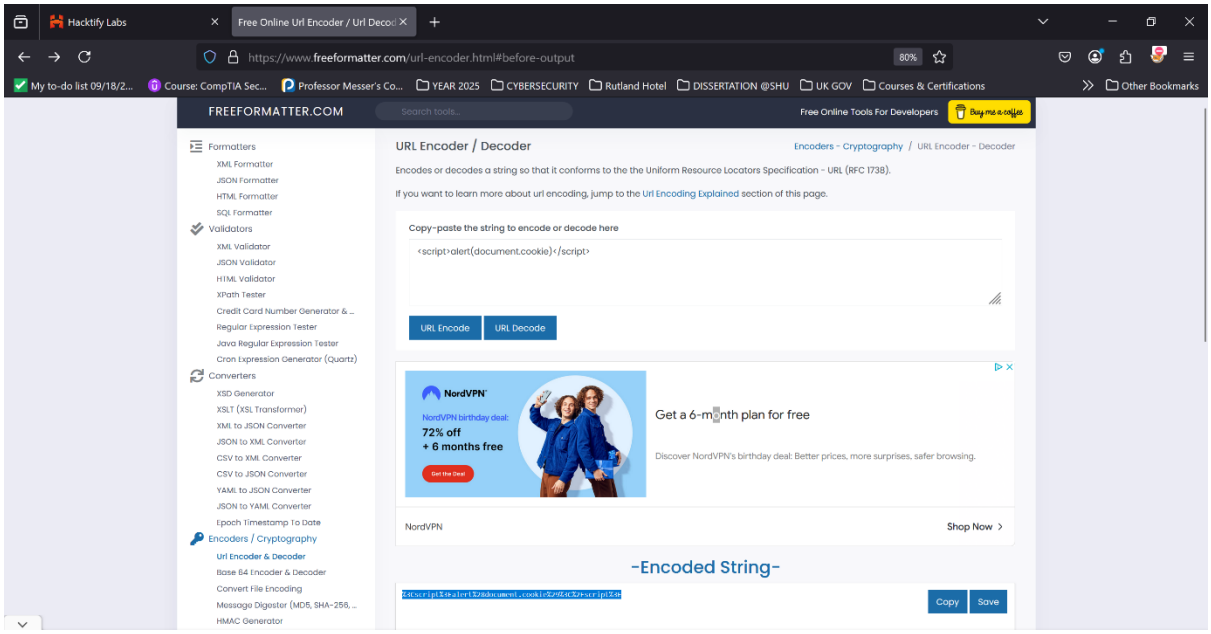
2.7. Encoding is the key?

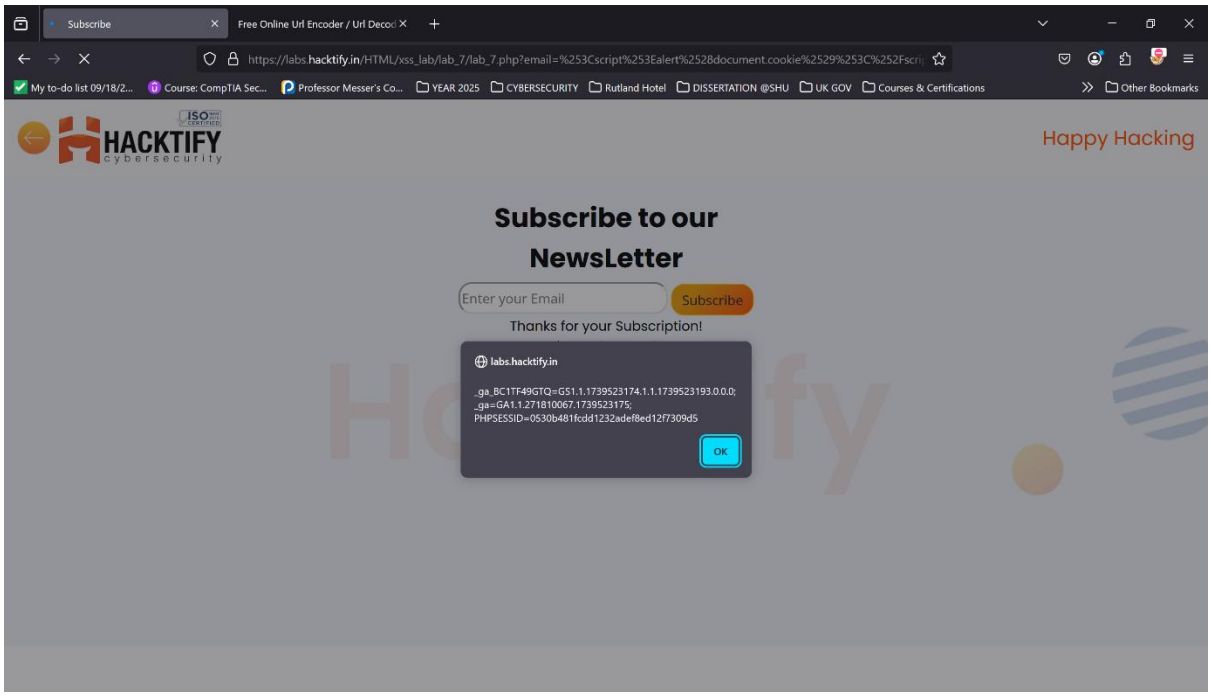
Reference	Risk Rating
Sub-lab-7: Encoding is the key?	**Medium**
Tools Used	
Web Browser, Online URL Encoder is used to find the vulnerability.	
Vulnerability Description	

This is a Reflected Cross-Site Scripting (XSS) vulnerability that occurs due to improper input validation and sanitization in the newsletter subscription form. The application accepts URL-encoded JavaScript code in the email parameter and executes it when decoded. The payload (%3Cscript%3Ealert%28document.cookie%29%3C%2Fscript%3E) when decoded becomes <script>alert(document.cookie)</script>, which executes and displays the user's cookies, demonstrating code execution capabilities.
How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/xss_lab/lab_7/index.php
Consequences of not Fixing the Issue
• **Session Hijacking** - Attackers can steal user session cookies and impersonate legitimate users • **Data Theft** - Malicious scripts can access and exfiltrate sensitive user information • **Account Compromise** - Attackers could perform unauthorized actions on behalf of the victim • **Malware Distribution** - The vulnerability could be used to redirect users to malicious sites or download malware
Suggested Countermeasures
• Implement proper input validation and sanitization using security libraries like DOMPurify or OWASP ESAPI • Enable Content Security Policy (CSP) headers to restrict script execution from unauthorized sources • HTML-encode all user-supplied data before rendering it in the response
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html https://www.sans.org/blog/cross-site-scripting-explained-xss/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab





2.8. XSS with File Upload (file name)

Reference	Risk Rating
Sub-lab-8: XSS with File Upload (file name)	**Low**
Tools Used	
Burp Suite Community Edition v2025.1.1, Web Browser are used to find this vulnerability.	
Vulnerability Description	
This is a stored XSS vulnerability in the file upload functionality. The application allows users to upload files without properly sanitizing the filename. When a file with a name containing JavaScript code (like	

<img src=x onerror=confirm(1)>) is uploaded, the application stores and later renders this filename as HTML rather than treating it as plain text. This allows JavaScript in the filename to be executed in the context of other users' browsers when they view the uploaded file details.
How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/xss_lab/lab_8/lab_8.php
Consequences of not Fixing the Issue
• Unauthorized access to user accounts through cookie theft • Session hijacking allowing attackers to impersonate legitimate users • Data theft from the application through DOM manipulation • Distribution of malware or phishing content to application users
Suggested Countermeasures
• Implement strict input validation and sanitization for filenames, rejecting any that contain special characters like <, >, ", ', etc. • Apply context-appropriate output encoding when displaying filenames (HTML-encode when displaying in HTML context) • Implement Content Security Policy (CSP) headers to restrict execution of inline scripts and limit sources of executable content.
References
https://owasp.org/www-community/attacks/xss/ https://portswigger.net/web-security/cross-site-scripting/stored https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/79.html

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

Free Online Url Encoder / Url Deco... | Upload a File

https://labs.hacktify.in/HTML/xss_lab/lab_8/lab_8.php

My to-do list 09/18/2... | Course: CompTIA Sec... | Professor Messer's Co... | YEAR 2025 | CYBERSECURITY | Rutland Hotel | DISSERTATION @SHU | UK GOV | Courses & Certifications | Other Bookmarks

HACKTIFY cybersecurity

Happy Hacking

Upload a File

Browse... test.html

File Upload

© Copyrights 2021 Hacktify Cybersecurity All rights reserved

Burp | Project | Intruder | Repeater | View | Help

Burp Suite Community Edition v2025.1.1 - Temporary Project

Dashboard | Target | Proxy | Intruder | Repeater | Collaborator | Sequencer | Decoder | Comparer | Logger | Organizer | Extensions | Learn | Settings

Intercept | HTTP history | WebSockets history | Match and replace | Proxy settings

Intercept on | Forward | Drop | Request to https://labs.hacktify.in:443 [162.0.229.223] | Open browser

Time	Type	Direction	Method	URL	Status code	Length
06:13:06 19 F...	HTTP	→ Request	POST	https://labs.hacktify.in/HTML/xss_lab/lab_8/lab_8.php		

Request

Pretty | Raw | Hex

```
7  Accept-Encoding: gzip, deflate, br
8  Content-Type: multipart/form-data; boundary=----geckoformboundary1a67ee7cbd22510d7664aa0049e72fb5
9  Content-Length: 348
10 Origin: https://labs.hacktify.in
11 Referer: https://labs.hacktify.in/HTML/xss_lab/lab_8/lab_8.php
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19
20 ------geckoformboundary1a67ee7cbd22510d7664aa0049e72fb5
21 Content-Disposition: form-data; name="image"; filename="test.html"
22 Content-Type: text/html
23
24 <script>alert(1)</script>
25
26 ------geckoformboundary1a67ee7cbd22510d7664aa0049e72fb5
27 Content-Disposition: form-data; name="upload"
28
29
30 ------geckoformboundary1a67ee7cbd22510d7664aa0049e72fb5--
```

Search | 0 highlights

Inspector

Selection 11 (0xb)

Selected text: "test.html"

Decoded from: Select

"test.html"

Cancel | Apply changes

Request attributes 2

Request query parameters 0

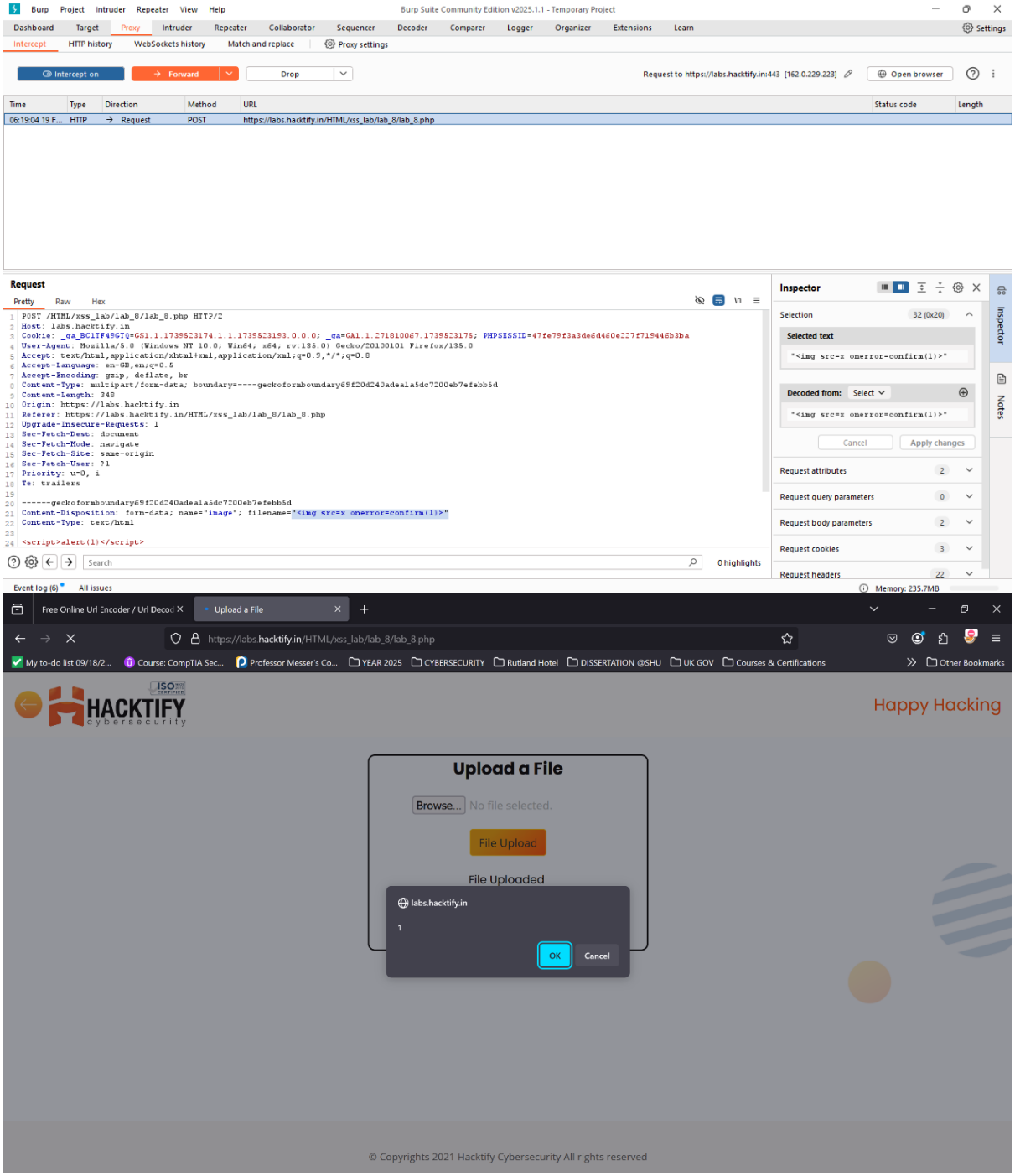
Request body parameters 2

Request cookies 3

Request headers 22

Inspector | Notes

Event log (6) | All issues | Memory: 233.3MB



2.9. XSS with File Upload (File Content)

Reference	Risk Rating
Sub-lab-9: XSS with File Upload (File Content)	**Low**
Tools Used	
Burp Suite Community Edition v2025.1.1, Web Browser is used to find this vulnerability.	
Vulnerability Description	
The application allows users to upload HTML files containing malicious JavaScript code without proper validation or sanitization. The vulnerability exists in the file upload functionality where the application	

accepts HTML files and executes embedded JavaScript code. Two successful payloads were demonstrated:

1. Basic XSS test: <script>alert(1)</script>
2. Cookie theft: <script>alert(document.cookie)</script> The application processes these files and executes the JavaScript code in the context of user's browser.

How It Was Discovered

Manual Analysis

Vulnerable URLs

https://labs.hacktify.in/HTML/xss_lab/lab_9/lab_9.php

Consequences of not Fixing the Issue

1. **Session Hijacking**: Attackers can steal user session cookies and impersonate legitimate users
2. **Data Theft**: Malicious scripts can access and exfiltrate sensitive user information
3. **Phishing Attacks**: Attackers can inject convincing forms or overlays to steal credentials
4. **Client-side Code Manipulation**: Attackers can modify webpage functionality and behavior

Suggested Countermeasures

- Implement strict file type validation and only allow secure file formats (e.g., .pdf, .jpg, .png) while explicitly blocking HTML and other script-capable formats.
- Apply Content-Type validation and Content-Disposition headers to prevent uploaded files from being executed as HTML/JavaScript.
- Implement Content Security Policy (CSP) headers to restrict execution of inline scripts and define trusted sources.

References

https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
https://portswigger.net/web-security/cross-site-scripting
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
https://cwe.mitre.org/data/definitions/434.html

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

Burp Project Intruder Repeater View Help — Burp Suite Community Edition v2025.1.1 - Temporary Project

Dashboard | Target | Proxy | Intruder | Repeater | Collaborator | Sequencer | Decoder | Comparer | Logger | Organizer | Extensions | Learn | Settings

Intercept | HTTP history | WebSockets history | Match and replace | Proxy settings

Intercept on | Forward | Drop | Request to https://labs.hacktify.in:443 [162.0.229.223] | Open browser

Time	Type	Direction	Method	URL	Status code	Length
04:19:12 19 F...	HTTP	→ Request	POST	https://labs.hacktify.in/HTML/xss_lab/lab_9/lab_9.php		

Request — Pretty | Raw | Hex

```
4  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:135.0) Gecko/20100101 Firefox/135.0
5  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6  Accept-Language: en-GB,en;q=0.5
7  Accept-Encoding: gzip, deflate, br
8  Content-Type: multipart/form-data; boundary=----geckoformboundarye2fcf56cd6025f6f765089c12586eded
9  Content-Length: 348
10 Origin: https://labs.hacktify.in
11 Referer: https://labs.hacktify.in/HTML/xss_lab/lab_9/lab_9.php
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19
20 ------geckoformboundarye2fcf56cd6025f6f765089c12586eded
21 Content-Disposition: form-data; name="image"; filename="test.html"
22 Content-Type: text/html
23
24 <script>alert(1)</script>
25
26 ------geckoformboundarye2fcf56cd6025f6f765089c12586eded
27 Content-Disposition: form-data; name="upload"
```

Inspector

Selection 25 (0x19)

Selected text: `<script>alert(1)</script>`

Decoded from: Select: `<script>alert(1)</script>`

Cancel | Apply changes

Request attributes 2
Request query parameters 0
Request body parameters 2
Request cookies 3
Request headers 22

Search | 0 highlights

Event log (6) | All issues | Memory: 220.9MB

Upload a File | Free Online Url Encoder / Url Decod

https://labs.hacktify.in/HTML/xss_lab/lab_9/lab_9.php

My to-do list 09/18/2... | Course: CompTIA Sec... | Professor Messer's Co... | YEAR 2025 | CYBERSECURITY | Rutland Hotel | DISSERTATION @SHU | UK GOV | Courses & Certifications | Other Bookmarks

HACKTIFY cybersecurity — Happy Hacking

Upload a File

Browse... test.html

File Upload

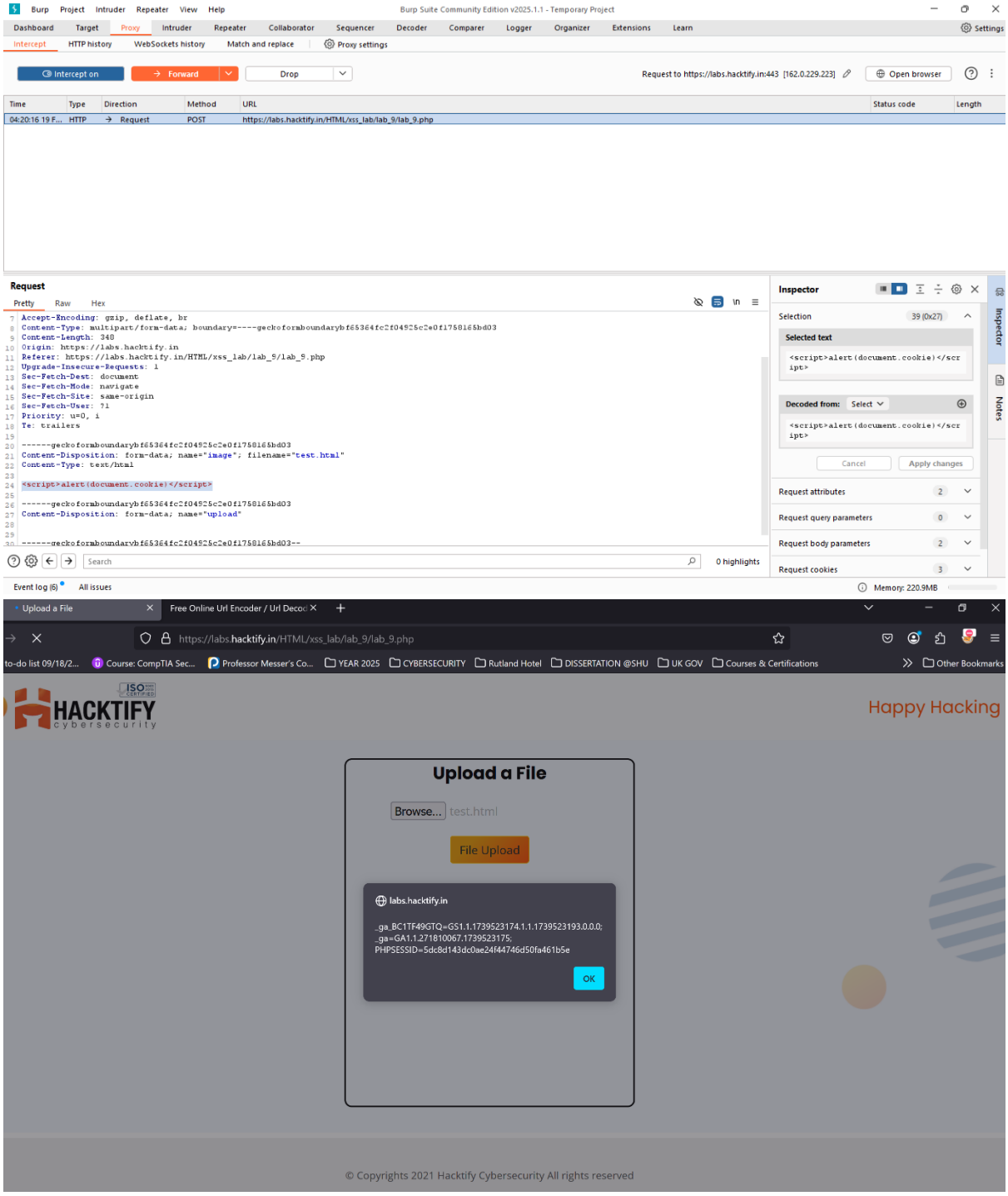
File Uploaded Named test.html

labs.hacktify.in

1

OK

© Copyrights 2021 Hacktify Cybersecurity All rights reserved



2.10. Stored Everywhere!

Reference	Risk Rating
Sub-lab-10: Stored Everywhere!	**Low**
Tools Used	
Web Browser is used to find this vulnerability.	
Vulnerability Description	
This is a stored XSS vulnerability affecting the user profile form on Hacktify's lab web environment. The application fails to properly sanitize user input in the first name, last name, and email fields. The payloads	

inserted (">fname<IMG SRC=javascript:alert(1)>, etc.) break out of the existing HTML context and inject malicious JavaScript that executes when the page loads. The fact that the payload remains visible in the form fields suggests successful injection, where the application stores these malicious scripts in its database and serves them to any user who views the profile page.

Payloads: ">fname<IMG SRC=javascript:alert(1)>
">lname<IMG SRC=javascript:alert(2)>
">email<IMG SRC=javascript:alert(3)>@gmail.com

How It Was Discovered

Manual Analysis

Vulnerable URLs

https://labs.hacktify.in/HTML/xss_lab/lab_10/lab_10.php

Consequences of not Fixing the Issue

1. **Session Hijacking** - Attackers can steal users' session cookies, allowing unauthorized access to accounts
2. **Credential Theft** - Malicious scripts can create fake login forms to harvest credentials
3. **Data Exfiltration** - Attackers can access and extract sensitive user information displayed on pages
4. **Website Defacement** - Malicious actors could modify page content, damaging the organization's reputation

Suggested Countermeasures

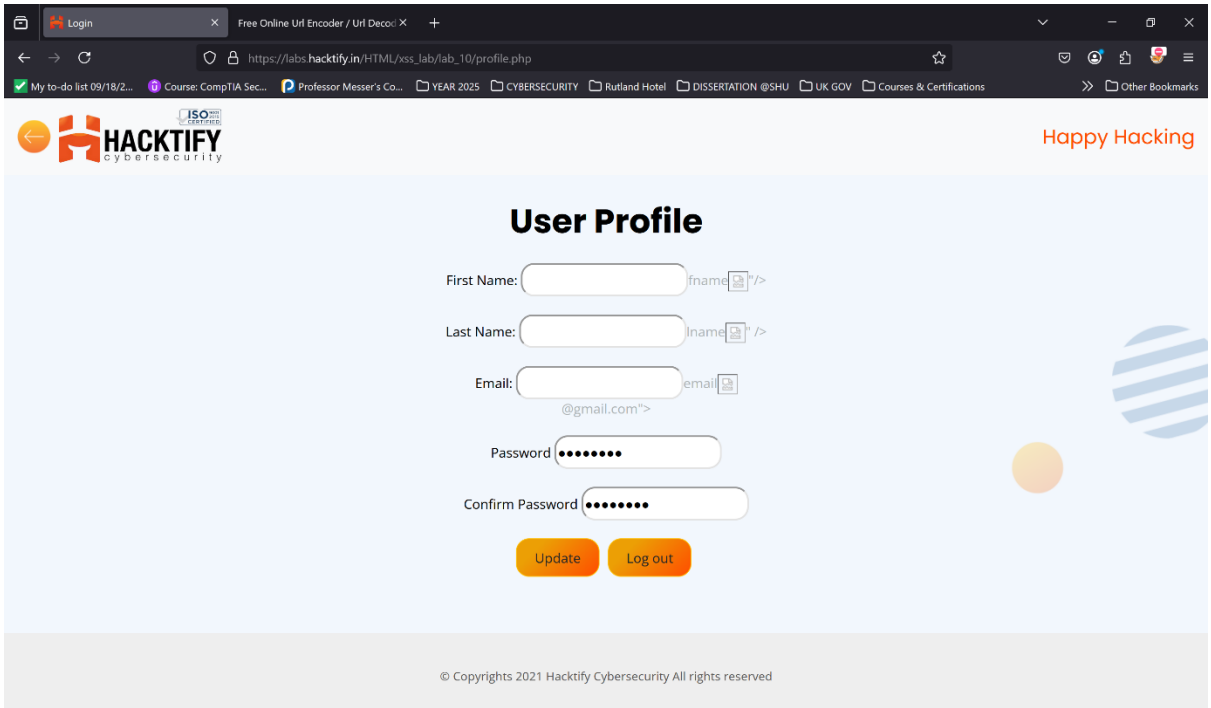
- Implement proper input validation and output encoding - Use context-appropriate encoding functions (HTML, JavaScript, CSS) to neutralize special characters.
- Implement Content Security Policy (CSP) headers to restrict execution of scripts from unauthorized sources.
- Use framework-specific security features like React's JSX escaping or Angular's automatic sanitization to prevent script injection.

References

https://owasp.org/www-community/attacks/xss/
https://portswigger.net/web-security/cross-site-scripting
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



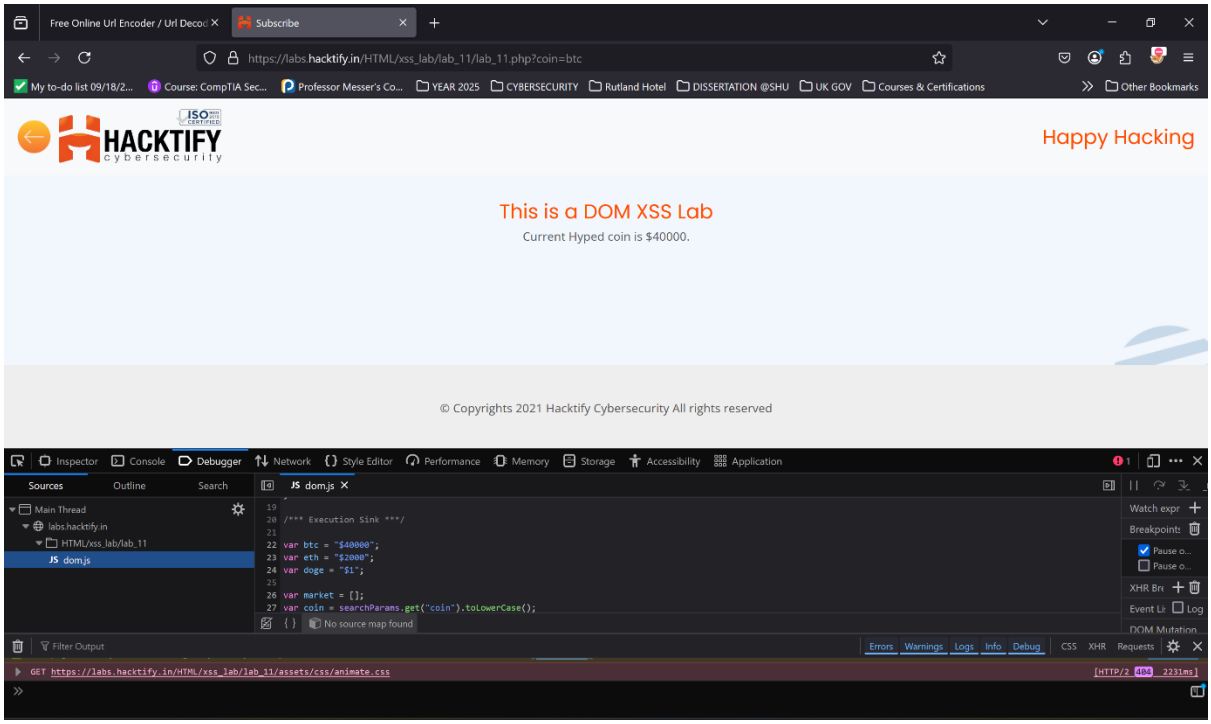
2.11. DOM's are love!

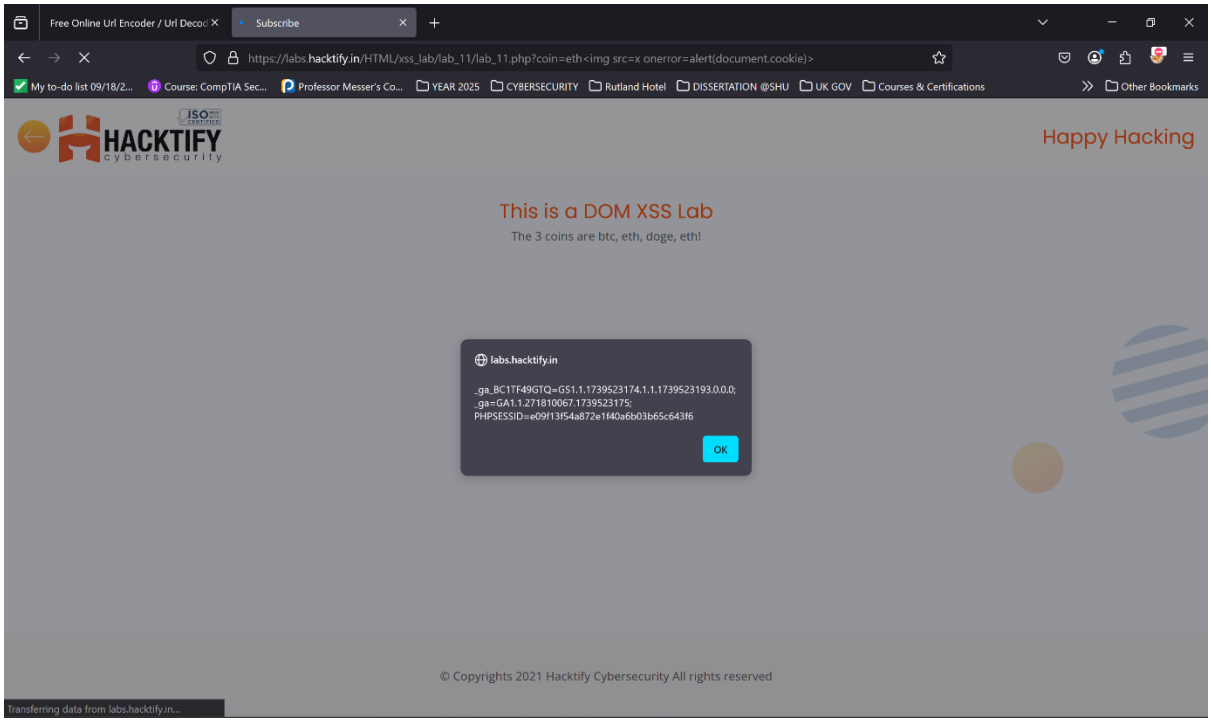
Reference	Risk Rating
Sub-lab-11: DOM's are love!	**High**
Tools Used	
Web Browser “Inspector” are used to find this vulnerability.	
Vulnerability Description	
This is a DOM-based XSS vulnerability where user input from the URL parameter coin is directly processed by client-side JavaScript without proper sanitization. The code in dom.js shows the application retrieves the parameter value using searchParams.get("coin") and uses it without validation. When a malicious value like eth<img src=x onerror=alert(document.cookie)> is provided, the browser executes the JavaScript in the onerror attribute, leading to the theft of cookies as shown in the alert dialog.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/xss_lab/lab_11/lab_11.php	
Consequences of not Fixing the Issue	
1. **Session hijacking** - attackers can steal user cookies and impersonate legitimate users. 2. **Data theft** - sensitive information stored in cookies can be compromised. 3. **Malicious actions** - attackers can perform unauthorized actions on behalf of users. 4. **Phishing attacks** - attackers can inject convincing UI elements to harvest credentials.	
Suggested Countermeasures	
1. Implement input validation and sanitization for the coin parameter before processing it in JavaScript. 2. Use Content Security Policy (CSP) headers to restrict execution of inline scripts.	

3. Encode output when rendering user-controlled data to the DOM using functions like textContent instead of innerHTML.
References
OWASP DOM Based XSS Prevention Cheat Sheet PortSwigger - DOM-based XSS MDN - Content Security Policy OWASP XSS Filter Evasion Cheat Sheet

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab




Free Online Url Encoder / Url Decod
Subscribe
https://labs.hacktify.in/HTML/xss_lab/lab_11/lab_11.php?coin=eth<img src=x onerror=alert(document.cookie)>
My to-do list 09/18/2...
Course: CompTIA Sec...
Professor Messer's Co...
YEAR 2025
CYBERSECURITY
Rutland Hotel
DISSERTATION @SHU
UK GOV
Courses & Certifications
Other Bookmarks
HACKTIFY
cybersecurity
Happy Hacking
This is a DOM XSS Lab
The 3 coins are btc, eth, doge, eth!
labs.hacktify.in
_ga_BC1TF49GTQ=GS1.1.1739523174.1.1.1739523193.0.0.0;
_ga=GA1.1.271810067.1739523175;
PHPSESSID=e09f13f54a872e1f40a6b03b65c643f6
OK
© Copyrights 2021 Hacktify Cybersecurity All rights reserved
Transferring data from labs.hacktify.in...